

# MiniSocial Project Overview

## Project Description

**MiniSocial** is a lightweight social networking application designed to enhance social interactions in a collaborative environment. The application allows users to connect, share ideas, and create engaging content. Users can register, maintain profiles, post updates and send friend requests. The framework is built using a clean, modular Java EE architecture, utilizing EJBs for business logic, JPA for persistence, JTA for transaction management, JAX-RS for RESTful APIs, JAAS for role-based security, and JMS for asynchronous messaging and notifications.

## Cheating Policy

- **Cheaters will be given a NEGATIVE grade and no excuses .**
- **Cheating means submitting a code you haven't written.**
- **“Not your Code” means: a friend code, a code generated from any AI tool, a code taken from a public github repo, or any other resource.**

## Project Deadline

- **Submission Date:** May 11th, 11:55 PM
- **This is a Strict Deadline that won't be changed.**

## Project Organization and Team Collaboration

- **File Naming Convention:** Upload your project as a ZIP file with the format `GroupNumber_ID1_ID2_ID3_ID4_TAName.zip`.
- **Team Size:** Maximum of **3-4 students** per team **from the same lab**.
- The Project worths **150 points (15 Marks)**.
- **Collaboration:** Ensure equal distribution of workload among team members. Each member should understand the overall project and be able to work on any component.
- You have to create a **postman collection** that includes all the API with ready request bodies and parameters. This collection will be evaluated at the discussion time. This will be a part of the grading criteria
- You need to implement your system **clean project architecture** to separate the project layers in different packages this point will be part of the grading criteria.
- **Note that: any violation in the assignment submission will cost you 0.5 mark of the assignment grade (like submitting more than one file, not following the naming convention, using different file extension)**

# Core Features

## 1. User Management [10 Points]

### 1.1 User Registration:

- Users can register for an account with a valid email address and password.

### 1.2 User Login:

- Users can authenticate themselves and log in to access the application.

### 1.3 Profile Management:

- Users can update their profile information, including their name, bio, email, and password.
  - A user chooses a role (user or admin) during profile creation.
- 

## 2. Connection Management (Social Networking Feature) [30 Points]

### 2.1 Friend Requests:

- Users can send and receive friend requests to connect with other users.

### 2.2 Manage Connections:

- Users can accept or reject pending friend requests.

### 2.3 View Connections:

- Users can view all their friends and their profiles.
- 

## 3. Post Management [40 Points]

### 3.1 Create Posts:

- Users can create status updates to share with their friends, including attaching images or links.

### 3.2 Feed Retrieval:

- Users can view a timeline feed that shows their posts and those of their friends.

### 3.3 Edit a post

- Users can update/delete a pre-posted post.

### 3.4 Engagement:

- Users can like and comment on posts made by friends, facilitating interaction.
- 

## 4. Groups Management [40 Points]

### 4.1 Create a Group [5 Points]

- Any user can create a new group (e.g., "Java Enthusiasts", "Book Club").
- A group has: Name, Description, Creator (Group Admin), List of members
- A group can be open or closed (requires admin permission to join).

### 4.2 Join & Leave Groups [15 Points]

- Users can request to join a group (pending approval), or Join directly if it's open.
- Group admin can approve or reject membership requests.
- Notifications **sent via JMS** when a member joins or gets approved.

### 4.3 Post in Group [10 Points]

- Members can create group-specific posts.
- Posts are only visible to other members of that group.

### 4.4 Group Roles & Permissions [10 Points]

- Group creator becomes the **Group Admin**.

- **Admin can:**
    - ✓ Promote other users to be admins
    - ✓ Remove posts or users from the group
    - ✓ Delete the group
  - Role-based logic enforced with **JAAS** or **custom EJB checks**.
- 

## 5. [Messaging Module] Notifications via JMS [15 Points]

- **Real-Time Notifications:**
    - Users receive notifications for events such as:
      - When a friend request is received.
      - When a friend likes or comments on their posts.
      - When a comment is made on their posts.
      - When Joins/Leaves a group.
  - **Notes:**
    - You must create a standardized event object capable of accommodating various event types.
    - Verification will involve utilizing a message-driven bean or publish-consumer models to print the received event. The event should be a JSON object containing task updates, event names, and other relevant information.
- 

## 6. Authentication and Security [ 15 Points]

- **Role-Based Access Control:**
    - Implement **JAAS** for role-based access:
      - **Admin Role:** Full access for user and content management. Only admins can apply CRUD operations to other users posts. Also **Check the Group Admin Permissions**.
      - **Regular User Role:** Limited access to create posts, manage friends, and view timelines.
  - **Access Restrictions:**
    - API endpoints for user management and admin functionalities are restricted to users with the Admin role. Unauthorized access attempts lead to a **403 Forbidden** error.
- 

## Bonus Features (Optional for all teams)

### 1 Media Attachments [2 Points]

- Allow users to post with image URLs.

### 2 User Search & Friend Suggestions [5 Points]

- Search users by name or email.
- Suggest friends based on mutual connections.

### 3 Activity Log [5 Points]

- Show a log of recent actions (posted, liked, friend added).
- Use **JMS** to queue and store logs asynchronously.

---

## Tech Stack Overview

- **JPA:** Manage data entities such as User, Post, FriendRequest, and Group.
- **EJBs:** Stateless beans that handle various services (e.g., UserService, PostService, GroupService).
- **JAX-RS:** RESTful APIs for interaction with the client-side application.
- **JTA:** Ensure data integrity through transaction management.
- **JAAS:** Security layer for user authentication and role management.
- **JMS:** Messaging system to handle notifications and asynchronous events.

---

## API References (For testing purposes)

The following are some samples for testing some APIs to guide you of how the output should look like.

### User Management APIs

#### 1. **Name:** User Registration

**URL:** /api/register

**Request Body:**

```
json
{
  "email": "user@example.com",
  "password": "UserPassword123",
  "name": "John Doe",
  "bio": "Hello! I'm a new user.",
  "role": "user" // or "admin"
}
```

**Response:**

**Status:** 200 Created (If registration is successful)

**Status:** 400 Bad Request (If input validation fails)

**Body:**

```
json
{
  "message": "User registered successfully."
}
```

---

#### 2. **Name:** User Login

**URL:** /api/login

**Request Body:**

```
json
{
  "email": "user@example.com",
  "password": "UserPassword123"
}
```

**Response:**

**Status:** 200 OK (If authentication is successful)

**Status:** 401 Unauthorized (If credentials are incorrect)

**Body:**

```
json
{
  "message": "Login successful.",
  "token": "JWT-TOKEN"
}
```

---

**3. Name:** Update User Profile

**URL:** /api/users/{user\_id}/update

**Request Body:**

```
json
{
  "name": "John Doe",
  "bio": "Updated bio.",
  "email": "newemail@example.com",
  "password": "NewPassword123"
}
```

**Response:**

**Status:** 200 OK (If profile is updated)

**Status:** 404 Not Found (If user doesn't exist)

**Body:**

```
json
{
  "message": "Profile updated successfully."
}
```

---